

A 'micro' case study

- This is a case study performed at a 'micro' level
 - equivalent to about a 'day' of Testing
- Lessons and observations are extrapolated to a 'macro' level
- Means that:
 - not all Testing Lessons will be discussed
 - the lessons mentioned, have context

Which App to choose

- Tracks
- <https://www.getontracks.org/>
- Web App, JS, API, DB
- Offers a lot of scope for testing

contexta

☆ □ ▾ contexta todo

contextb

☆ □ ▾ uncomplete

contextc

☆ □ ▾ contextc todo

con

☆ □ ▾ Action

Completed actions in this context

Show all

☆ [x] ▾ 27/11/2019 contextb todo (contextb)

Send feedback on 2.3.0: [Bugs](#) | [Wiki](#) | [Mailing List](#) | [Website](#) | [Contribute](#) | [Mobile Site](#)
 Tracks Appliance – Powered by [TurnKey Linux](#)

Hide form | Add multiple next actions

Description

Notes

Project

Context

contexta

Tags (separate with commas)

Due

Show from

Depends on

✓ Add action

Active projects (0)

- None

Active contexts (4)

- contexta (1 action)
- contextb (1 action)
- contextc (1 action)
- con (1 action)

Lesson

- Normally we don't get to choose
 - Or do we?
- We need to allocate staff based on skills and experience,
 - not just availability
- If we don't understand the technology
 - we need to manage that as a risk

Environment

Could use:

- Docker
- Local install
- VM
- Cloud hosted

Each has different risks and constraints.

My Environment Requirements

- proxy traffic
- possibly view DB
- low impact on my machine

Knowledge Constraints

- My Tech knowledge of Docker
- Lower Docker knowledge than VM
- Know how to configure and access DB on VM
- Chosen VM <https://www.turnkeylinux.org/tracks>
- Issue... old version of tracks
 - not a concern for this talk

Lesson Environment Impacts Testing

- Do staff have permissions?
 - to observe, interrogate, manipulate?
- Does environment match deployment?
- Does release process match live?
- Version management
 - same as live?
 - can do upgrades?

Install Session

```
# Install Session
```

```
20191125 10:00
```

```
* download vm  
* install tracks  
* 192.168.1.36  
* R00troot
```

```
10:30
```

Environment Ready

- Installed VM
- Login as Admin
- Is tracks ready?
 - Don't know
 - No health check or status to inform me

Lesson

- Automated Execution can help derisk deployment and startup
- Without this, the first 'test' sessions are 'health check' sessions
- Otherwise there is a risk of wasted time but finding out too late

Health Check

- CRUD
- Create, Read, Update, Delete
- Main Areas
 - Keep it simple, create a todo

Health check Session

20191125 - 14:00

- Create from front page

- [Add Action] - blank, XHR, error message rendered via JavaScript interpreted from Response
- [Add Action] - Desc 'a' - error, requires Context
- [Add Action] - Desc 'a', Context 'c' - created

- Read by changing between tabs

- home -> projects -> home and todo rendered

- Amend from front page

- a -> aa

Delete from front page

- create 'b' in 'c' context, for deletion
 - marked as completed anticipating delete option but moved to completed
 - Note - completed action can be dragged ?To where, for what purpose?

contexta

     contexta todo

contextb

    uncomplete

contextc

    contextc todo

con

    Action

Completed actions in this context

    27/11/2019 contextb todo (contextb)

Lessons

- All testing is exploratory, observation and exploration during health check
- Exploration boosts (and is constrained by) technical knowledge of the app
- Time Box Testing
- I need to have screenshot tools to help gather evidence quickly during testing
- Make Notes Prior (Plan) (AIMs)
- Make More Notes During (Actual)
- Gather Evidence

What do I test?

- Requirements
 - Often driven by 'stories' on Agile
 - What a User wants to achieve
 - Why they want it
- How it has been implemented
 - Functional Description
 - Technical Risk
- Decisions & Constraints made during development
- Priorities and Agreed Risk Areas

In-Sprint, Story and System Testing

In-sprint testing

- constrained to 'work done' scope
- artificially constrained - no error handling, no security etc.
- need to track these 'ommissions' as coverage gaps for later testing
- often testing 'bits' of stories

Stories

- Complete chunk of work
- Sometimes span sprints
 - 'not complete' but still requiring testing
- Acceptance Criteria testing needs to be revisited
- Automated Execution to cover criteria can help
- Going 'beyond' the story often needs to wait till story is 'complete'
- Stories often tested in isolation - Automated Execution in a CI can help

System Testing is a vague term

- I'm meaning 'system as a whole'
- Stories in combination i.e. functional flows that span stories
- Stories in different orders
- Going beyond the story Acceptance Criteria
- Often 'neglected' in Agile Projects

Tracks v2.3.0 Release Notes

<https://www.getontracks.org/news/comments/release-2.3.0/>

- Changes
- Bug Fixes

Could review code committed since previous release.

Tracks 2.3.0 released

Tracks 2.3.0 has been released! This version brings a few small changes to the UI, many bug fixes, and some major under-the-hood upgrades and refactors.

Tracks is now running on Rails 4.1. Ruby 1.8.7 is no longer supported, but this release of Tracks is fully tested on Ruby 1.9.3, 2.0.0, and 2.1.

Other changes:

- You can select to group todos on the home page by context or by project (using the view menu). This also works for tag page, the project page, the tickler and the context page
- You can now change the state of a context to closed
- Czech locale has been renamed from cz to cs to follow ISO standards
- Added Russian locale (@AlexStein)
- The toggle-notes and toggle-collapsed-containers have been moved into the view menu.
- Numerous bug fixes

You can download the release [here](#). Upgrade instructions can be found [in the manual](#).

Get Tracks



[Tracks 2.4.1 release notes](#)

[Report a bug](#)

Github Repos

[tracks](#)

Tracks is a GTD™ web application, built with Ruby on Rails

[tracksapp.github.io](#)

The official Tracks website, hosted on GitHub Pages

[.github](#)

Default community health files for Tracks

[@TracksApp](#) on Github

Community

[Gitter chat](#)

[Mailing list](#)

Taking a System and Beyond View

I will assume that testing has been done on the stories and all stories complete in the sprint and release.

I will take a System view and ignore specific 'sprint', 'story' or 'release' requirements. View functionality as a whole.

I will not review automated execution. Risk of duplicated effort and scope.

Q: How does a manager add value on an Agile Project?

A: By looking at the project holistically.

- Teams are focussed on the functionality that they are working on.
- Teams often don't have time to look beyond.
- Teams are often not encouraged to think beyond.

Process risk, can be used to drive testing. Look where others don't.

Planning Session

- Making the decision about what to test is also a 'session'
- It is an exploratory process of weighing up factors, asking questions and making decisions
- It can be tracked and planned like any other activity
- It is important not to go into too deeply because we are working with unknowns
- But if we "don't plan" because we are Agile, or Exploratory we are losing an opportunity to effectively target our testing.

Planning session

20191126 - 13:00

- release notes
- commits?
- defects?
- general functionality?
- assume team tested, add value by going 'holistic'
- pick release note item and go beyond acceptance criteria

"You can now change the state of a context to closed"

Questions:

- What is a Context?
- What is Context State?
- What states are there?
- Is it a state machine?
- Are all states equally valid?
- Are there transition rules?

20191126 - 13:10

My Chosen Scope

From <https://www.getontracks.org/news/comments/release-2.3.0/>

"You can now change the state of a context to closed"

- You can select to group todos on the home page by context or by project (using the view menu). This also works for tag page, the project page, the tickler and the context page
- You can now change the state of a context to closed
- Czech locale has been renamed from cz to cs to follow ISO standards
- Added Russian locale (@AlexStein)
- The toggle-notes and toggle-collapsed-containers have been moved into the view menu.
- Numerous bug fixes

When I start testing, I don't know what that means yet.

I have to explore to find out.

Context State

"Charter"

"You can now change the state of a context to closed"

- What is a Context?
- What is Context State?
- What states are there?
- Is it a state machine?
- Are all states equally valid?
- Are there transition rules?

Recon Session

Does dragging between context states on GUI change state?

Tried - does not seem to. But only have 1 context. Create more and see if it makes a diff.

contexta, contextb, contextc, contextd

No - can't drag. Not sure why drag functionality is present.

Drag is used to re-order within a particular state.

Seem to be no state constraints i.e. can move from

active -> hidden -> closed seemingly in any order.

Formally create combos and try

- active -> hidden
- hidden -> closed

Recon Session

Findings:

- todos added to context
- context seems to move between state in any order (TODO: cover state table)
- constraint, cannot move context to closed if open actions
- but can add open actions to closed contexts
- all contexts are shown in auto complete drop down regardless of state (order controlled by Organize view)
- seems to be XHR updates but need to view traffic to be sure
- open screens do not update with state unless refreshed e.g. home, when open and move context to new state, does not show new state - can this be used to abuse state? e.g. drag todo as sub todo on a todo on a closed state and closed todo that has not refreshed yet?

Lessons Learned:

- Functional Testing is Functional Testing
- Architecture and technology changes observation ability
- My modelling and assessment of findings is limited by my ability to observe
- Choosing and Selecting Tooling is an evolving process based on the needs of the testing and the application.
- I don't 'start' with tools, I iterate towards tool usage as required.

Debrief Session

- Look at what we've done
 - Working exploratory means I have to 'step back' periodically
- Collate Notes
- Identify Issues, Todos, Raise Defects
- Feed into Planning Session
- Refine/Formalise Models - Modelling Session if 'large'

Aim: review notes, identify todos, issues, models, status, summarise

- Have installed software in VM, seems to work well enough for testing
- Gui rendering issues on pop up drop downs
 - in situ on screen
 - when dragging on screen
 - need to have screen capture tool ready to capture rendering issues as they vary between sessions
- Learned basic mandatory relationship between Action and Context
 - Action needs a Context
- Currently observing results on GUI, want to expand observation to traffic and DataBase.
- Currently manipulating through GUI, want to expand manipulation to API. Can then observe and interrogate through API.
- Context state interaction
 - Context can have states Open. Hidden. Closed

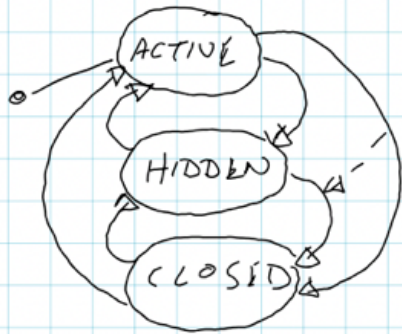
Lessons Learned

- I rely on linear text as much as possible
- text files, markdown, screenshots in `images` folder
- formatting, aesthetics, structure
 - can be added automatically and during debrief
- when working on own, this is indispensable
- can act as a 'status' report

Modelling Session

- "prep a more comprehensive coverage approach because I didn't 'test' this I did an initial exploration during recon but the coverage wasn't documented or thought through well enough."

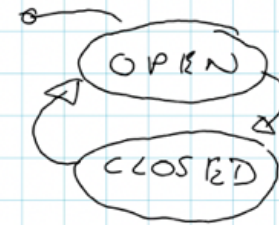
CONTEXT STATE



CONSTRAINT
CAN NOT MOVE
TO CLOSED IF

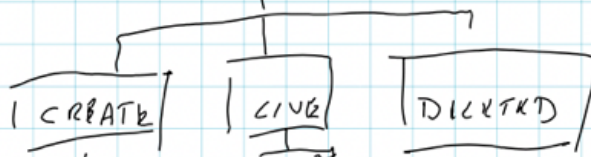
CONTEXT $\xleftarrow{\text{OPEN ACTION}}$
 ≥ 1

ACTION STATE

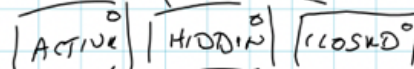


STATE SWITCH AS EVENTS

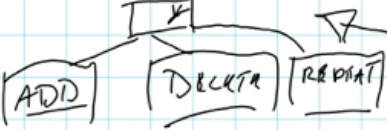
CONTEXT



ACTIVE



ACTIONS

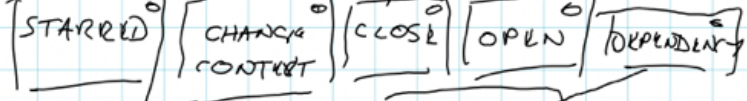
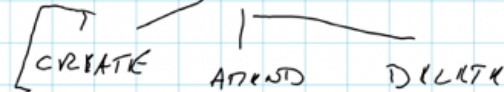


RELATIONSHIP

HAS ACTIONS STATE

DON'T
STATE
MAKE
A
DIFFERENCE
TO DELETE?

ACTION



+ -
DEPENDANT
UPON
ACTION

DEPENDENCY
BETWEEN
STATES
HAS DEPENDENTS
IS DEPENDENT ON

ACTION
STATE
OPEN
CLOSED

Lessons Learned

- Modelling is a key skill
- When our models are ambiguous it might mean we don't understand
- Simple concepts can expand into complex models
- Complex models have a high number of combinations
- Formal modelling often has tooling support
- Informal modelling can support exploration
- Don't leap to a diagrammer tool
 - pen, paper, smartphone camera
 - whiteboard -> smartphone camera
 - tech solution smartpen -> Evernote

Led to a Quick Planning Session

MOST IMPORTANT TO EXPLORE

CONTEXT - CLOSED

CONTEXT OPEN $\rightarrow$ CLOSED

CONTEXT CLOSED $\rightarrow$ OPEN

ACTION $\leq$ ACTION STATE

EVENT

- $\leq$ - CREATED $\left\{ \begin{array}{l} \text{NONE} \\ \text{VARIABLE} \end{array} \right.$
- CHANGE CONTEXT
- CLOSE
- OPEN
- DEPENDENCIES

If this was a real project:

- add this as notes in a Jira task
- add this to kanban board
- discuss with team
- possibly expand further and write up, if not doing immediately and model is complex

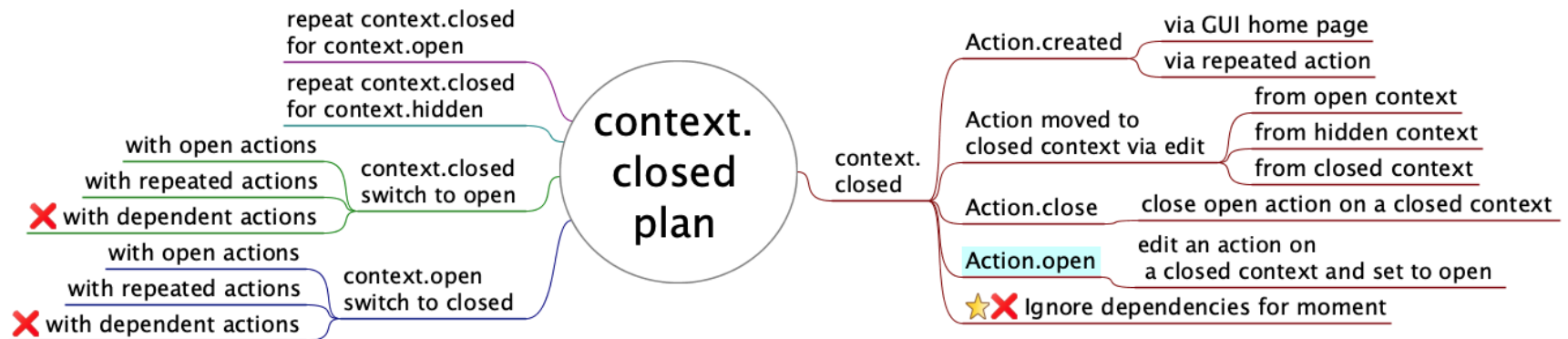
Lessons Learned

- Modelling and Planning are related
- Planning is prioritising parts of model coverage
- Having a representation of your model means it is easier to see, what else can be done later
- Initial plan helps maximise the effectiveness of the testing sessions
- Expand Plan as far as you need to to support the next session

Coverage Session

- Actual Testing
- Created a mindmap in advance to expand the plan
- kept notes as tested
- discovered some missing items from the 'plan' added them into my notes
- trying to keep additional exploration to a minimum – focus on 'coverage' of defined plan

Expanded Plan

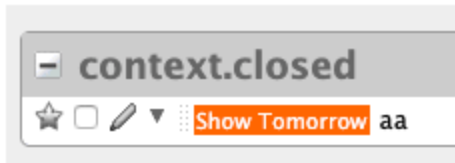


Coverage Session Notes

in error action (not plan) not planned,

set to Deferred - "aa"

success, now in 'tickler"



Context closed

Switch to Active with open actions - use "context.closed"

change state from context view

<http://192.168.1.36/contexts>

Context Saved

Lessons Learned

- Gathered more evidence when testing, than during recon
- Expand plans as required, when required
 - I use FreePlan because of build in scripting and easy file parsing
- Plan may not be complete
 - be aware of new opportunities to test
 - if in keeping with current aim then follow
 - if out of scope then note down idea as a todo and plan via debrief
 - keep notes of expanded extra paths followed
- tooling support for observation, interrogation. manipulation appropriate for scope and aim i.e. I didn't use any extra tools for testing, but I did for planning

Is it a real issue?

- Don't know
- On a real project I would not have pursued it so far, without finding that out first.
- It is the type of issue that would be found via exploratory processes

If it is an issue, when would it be found?

- Structured/Traditional testing
 - hopefully during a Q&A, clarification when writing 'cases/conditions' from the requirements
 - but it might take days/weeks to get to that point depending on process e.g.
 - analyse all test conditions and ideas against requirements and remove ambiguity - might find it within days
 - if test conditions, converted to cases, converted to scripts and work through requirements in sequence it might take weeks

If it is an issue, when would it be found?

- Agile
 - hopefully during a story understanding or development session
 - if not then, would it be found from following acceptance criteria?
 - probably not, it would be found in the exploratory testing 'around' the story

Note: questioning and clarifying during analysis is an Exploratory process. This can be found early, by exploring early.

Projects make decisions about defects

In this case it seems like a mismatch in states. The constraint is too easy to bypass if it is an important constraint e.g.

Instead of creating the Action on a closed context. I create it on an Active context, then amend the action to be on a closed context.

But, the impact in terms of system reliability, robustness, data integrity etc. is minor so I might not push hard on this, I might point out that it seems pretty inconsistent.

What Drives Tooling?

- Observation
- Interrogation
- Manipulation
- Admin
 - process support, Evidence Gathering, Note Taking, etc.

Observation

- seeing in real time
- possibility to quick catch issues
- supports monitoring e.g. alerts on errors in logs

Interrogation

- deep dive 'after'
- could be 'seconds' after, but still 'after'
- more data than can be absorbed quickly

Manipulation

- ability to change the system -defaults, static data
- repeat/amend requests
- use the API

Evidence Gathering

- recorded observations
- note taking

What Drives Tooling ... Needs of Testing

For Web

I can test 'blind' and trust the GUI or I can add tooling

- Dev Tools
- Proxy Tools

Both allow observation and interrogation. Proxy tool offers scope for easier longer term interrogation and manipulation for replay.

For Technology

Find tools that allow:

- Observation
- Interrogation
- Manipulation
- Support Scripting, extension, apis
- Record Evidence

(Technical) Exploratory Session

"Aim: Investigate the traffic mechanisms for creating actions and changing state on context - how far can we push this?"

Technical Exploratory Session Difference From Exploratory Session

- Technical Exploratory Session using observation at a technology level e.g. traffic, params, headers
- Manipulation at a technical level e.g. resend requests, DOM manipulation
- Testing often driven by Technical observation - what happens if I change that field? what if there are multiple fields?
- Bypass system flow and GUI, send messages directly
- Tools mandatory

Using

- Firefox (because I have fewer plugins to create noise in the Proxy)
- FoxyProxy plugin (configure use of ZAP Proxy)
- Owasp ZAP Proxy (Observe, Interrogate, Manipulate HTTP)
- Firefox Dev Tools (interrogate/manipulate DOM)
- TextEditor - view XML (Sublime Text)
- SnagIt - screenshots
- Evernote - not taking

Basic Process

- Work in GUI
- Observe impact via HTTP (I can see if JS or Server validation used)
- Interrogate HTTP Traffic
 - source of additional testing ideas
- Manipulate (resend) HTTP Traffic for message/param exploration
- Trigger interesting results via DOM manipulation in GUI
- very important to keep an eye on time and scope of testing

Notes Overview

- Install new version of ZAP Proxy - v2.8.0
- refreshed certificate in firefox - preferences, certificates, "trusted issuer" - choose 'options' in import to select
 - had to delete existing, before it all worked
 - ZAP hud started when I went to site. I just want basic proxy at the moment so configured to switch it off

12:05 - proxy ready

restarting session

On zap on Mac - tried to persist session in sub folder but it doesn't show folders - is that a bug or an odd mac thing after OS upgrade - not sure if session is actually being persisted - saving to ~/Documents/currentdate

- create a new Action on a closed context



Using 'new closed.context'

"create 1128 Action on closed context"

Notes Subset

- create a new Action on a closed context

Using 'new closed.context'

"create 1128 Action on closed context"

url encoded form post

POST <https://192.168.1.36/todos>

utf8=%E2%9C%93&authenticity_token=6zklFBkNF9u6Xax5qtJGE06YQfzzCV6NSp6%2B6l8%2Fm5c%3D&default_pr
object_name=&default_context_name=contexta&new_todo_starred=false&todo%5Bdescription%5D=create+1128+Acti
on+on+closed+context&todo%5Bnotes%5D=&project_name=&context_name=new+closed.context&initial_tag_list=&ta
g_list=&todo%5Bdue%5D=&todo%5Bshow_from%5D=&predecessor_input=&predecessor_list=&_source_view=todo&
_source_view=todo&_group_view_by=context&_tag_name=

what is default context name used for?

Looked in preferences and can see a default context for email - set to context.closed (not sure how that happened)

Can't see a setting for default context.

What if I pass in the message without a context_name but have the closed context as the default_context_name?

What I Found

- possible bug in ZAPProxy - session only savable to Documents, not sub folder
- possible Tracks bug - no error message from system when incorrect status used for context, the value is ignored, the user is told 200 OK, suggests Tracks 'trusts' the GUI so error handling might need more investigation across system
- Tracks XHR message responses which are error messages are 200 status rather than a 4xx status

What I Found

- many 'interesting things about tracks' that I might want to investigate later e.g. `_method=put` field in request different from HTTP verb, what is a "default context" and how is it set, Tracks responds with HTML and JS rather than a 'message' like JSON
- API vs GUI mismatch? ie a forms API and a JSON API
- limited in what I could observe because I did not include DB tools, had to use System exports to check data.

Tooling Benefits

- automatically record HTTP session
- easy replay for requests
- easy to view traffic without having to use dev tools
- use dev tools for DOM Inspection and manipulation
- use proxy for HTTP Observation, Inspection and Manipulation

Lessons Learned

- Tools can impact testing - lost time due to tool setup
- More information can mean more risk of going off charter or off time
- Test Ideas that are not possible without tooling (e.g. duplicate params, re-order params)
- Duplicate 'bugs' at GUI as well as HTTP level to demonstrate possibility e.g. amend DOM, not just HTTP
- Automated record keeping is a 'backup', I still copy HTTP request and URLs into my notes

What Did I Not Do/Show? – Automating

- likely factored into "Story" Testing
- start tactical, make it work, refactor to abstraction layers, become more strategic as required
- use default tools you know how to get value out of quickly

What Did I Not Do/Show? - Admin

- Admin Sessions
- Raising Defects
 - Use existing logs and evidence where possible.
 - Use logs to help recreate
- Clarifying Questions
- Communication
- Uploading Logs to Time Tracker
- keep as light weight as possible
 - use existing team tools, rather than adding 'test tools' - particularly on Agile - as much as possible



Testing

- 241 minutes (4 hours) spent Testing
- 93 minutes (1.5 hours) hands on
- 2-3+ hours additional admin if 'real' project

Much of Testing goes unnoticed and untracked.

Note taking is key to gathering evidence and making testing visible.

Bugs

Testing is not all about bugs.

- The most visible output, are bugs.
- Many projects only care about bugs.
- Testing is a process of deliberate exploration and coverage.
- Finding bugs is a side-effect of this process.

Tooling

Tooling comes from:

- need
 - observation
 - interrogation
 - manipulation
 - admin (reporting, management, etc.)
- technology
 - chosen project technology can aid/hinder tooling

Exploration And Coverage

We can explore at all points in our testing. Observation and reflection can identify new opportunities. New opportunities do not need to be followed immediately.

- The more we are focussed on coverage the more we constrain our exploration within the coverage.
- The more we are focussed on Exploration, the more our coverage has to be reverse engineered from our logs, evidence and notes.